Illustris/IllustrisTNG Data, the Converter, and the LaRT AMR Interface

Contents

1. Overview of Illustris and IllustrisTNG

The Illustris [1, 2] and IllustrisTNG [3, 4] projects are large-scale cosmological magneto-hydrodynamic simulations of galaxy formation and evolution. They use the AREPO moving-mesh code [5], which solves the Euler equations on an unstructured Voronoi tessellation that moves with the local fluid velocity.

The key difference from AMR codes (such as RAMSES) is that the spatial discretization is an unstructured Voronoi mesh rather than a structured octree. Each gas cell is a convex Voronoi polyhedron defined by a generator point; the cell positions, volumes, and all physical quantities are stored directly in HDF5 snapshot files.

1.1 Simulation Variants

The publicly available simulation suites include:

Suite	Runs	Box (cMpc)	Ref.
Illustris	Illustris-1, -2, -3	75	[1]
IllustrisTNG	TNG50-1 to -4	51.7	[3]
	TNG100-1 to -3	110.7	
	TNG300-1 to -3	302.6	

IllustrisTNG improved upon Illustris with updated AGN feedback (kinetic and thermal modes), refined supernova feedback, and inclusion of magnetic fields.

2. Data Format

Both Illustris and IllustrisTNG store data as HDF5 files containing raw snapshots, group catalogs, merger trees, and supplementary data catalogs. This section focuses on the snapshot format, which is the input to the LaRT converter.

2.1 Snapshot Structure

Each snapshot is split across multiple HDF5 chunk files. A single chunk file has the following top-level groups:

HDF5 Group	Contents
`Header`	Metadata: particle counts, box size, cosmological parameters, redshift, scale factor
`Parameters`	Simulation setup parameters (optional)
`Configuration`	Code configuration details (optional)
`PartType0`	Gas cells (Voronoi mesh elements)
`PartType1`	Dark matter particles
`PartType3`	Tracer particles
`PartType4`	Stars and wind particles
`PartType5`	Black holes

API cutout files (downloaded from the TNG web interface) have a similar structure but contain only particles/cells belonging to the requested halo or subhalo.

2.2 Header Attributes

The `Header` group contains the following attributes relevant to the converter:

Attribute	Type	Description
`BoxSize`	float64	Simulation box side length [ckpc/h]
`HubbleParam`	float64	Hubble parameter h (e.g. 0.6774)
`Redshift`	float64	Snapshot redshift z
`Time`	float64	Scale factor $a = 1/(1+z)$
`Omega0`	float64	Matter density parameter Ω_0
`OmegaLambda`	float64	Dark energy density parameter Ω_Λ
`NumPart_Total`	int32[6]	Total particle counts per type
`NumPart_ThisFile`	int32[6]	Particle counts in this chunk

2.3 Gas Cell Fields (`PartType0`)

Table 1 lists the `PartType0` datasets used by the converter, along with their Illustris code units and the conversion formulas to physical (CGS/kpc/km s^{-1}) units.

Table 1: Gas cell fields in `PartType0` relevant to LaRT. Comoving coordinates use scale factor a and Hubble parameter h from the snapshot header.

Dataset	Code Units	Physical Conversion
`Coordinates`	ckpc/h (comoving)	$x_{\rm phys}[\rm kpc] = x_{\rm code} \times a/h$
`Density`	$(10^{10} M_\odot/h)/(\rm ckpc/h)^3$	$\rho_{\rm cgs} = \rho_{\rm code} \times 10^{10} M_\odot h^2/(a\ \rm kpc_{cm})^3$
`Masses`	$10^{10} M_\odot/h$	$M[\rm g] = M_{\rm code} \times 10^{10} M_\odot/h$
`InternalEnergy`	$(\rm km/s)^2$	$T = (\gamma-1)\, u \times 10^{10}\, \mu/k_B$ (see §3.3)
`Velocities`	km $\sqrt{a}$ / s	$v_{\rm phys}[\rm km/s] = v_{\rm code} \times \sqrt{a}$
`GFM_Metallicity`	dimensionless	Total metal mass fraction Z (not relative to solar)
`NeutralHydrogen-Abundance`	dimensionless	Neutral hydrogen fraction $x_{\rm HI} \in [0, 1]$ (from TNG on-the-fly chemistry)
`ElectronAbundance`	dimensionless	$x_e = n_e/n_{\rm H}$
`Volume`	$(\rm ckpc/h)^3$	$V_{\rm phys}[\rm kpc^3] = V_{\rm code} \times (a/h)^3$
`StarFormationRate`	$M_\odot$/yr	Instantaneous SFR; cells with SFR > 0 are on the effective equation of state (see §4.)

2.4 Voronoi Mesh Geometry

Unlike AMR codes where cell geometry is defined by the octree level and position, Arepo cells are Voronoi polyhedra. The `Coordinates` field gives the Voronoi generator (control point) position, and the `Volume` field gives the cell volume directly. Cell faces and neighbor connectivity are *not* stored in the snapshot files; they must be reconstructed from the Delaunay triangulation if needed.

For the purposes of converting to an octree, the key insight is that the Voronoi tessellation defines a spatial partition: every point in the simulation domain belongs to exactly one cell, determined by which generator is closest. This property is exploited by the nearest-neighbor assignment in the converter (Section 5.4).

3. Unit Conversions

Illustris/TNG stores all quantities in **comoving coordinates with h-factors**. The converter transforms them to physical units suitable for LaRT.

3.1 Positions

$$x_{\rm phys}[\rm kpc] = x_{\rm code}[\rm ckpc/h] \times \frac{a}{h} \tag{1}$$

After conversion, coordinates are recentered so the galaxy center lies at the origin (centered convention, range $[-L/2, +L/2]$).

3.2 Density and Hydrogen Number Density

$$\rho_{\rm cgs}[{\rm g/cm^3}] = \rho_{\rm code} \times \frac{10^{10}\, M_\odot \times h^2}{(a \times {\rm kpc_{cm}})^3} \tag{2}$$

$$n_{\rm H}[{\rm cm^{-3}}] = \frac{\rho_{\rm cgs} \times X_{\rm H}}{m_{\rm H}} \tag{3}$$

where $X_{\rm H} = 0.76$ is the primordial hydrogen mass fraction and $m_{\rm H} = 1.6726 \times 10^{-24}$ g.

3.3 Temperature

Illustris stores the specific thermal energy u [$({\rm km/s})^2 = 10^{10}$ erg/g]. The temperature is computed from u and the electron abundance x_e:

$$\mu = \frac{4\, m_p}{1 + 3\, X_{\rm H} + 4\, X_{\rm H}\, x_e} \tag{4}$$

$$T[{\rm K}] = \frac{(\gamma - 1)\, u \times 10^{10}\, \mu}{k_B} \tag{5}$$

where $\gamma = 5/3$, $m_p = 1.6726 \times 10^{-24}$ g, and $k_B = 1.3807 \times 10^{-16}$ erg K^{-1}. When the `ElectronAbundance` field is unavailable, the converter assumes fully neutral gas ($x_e = 0$).

3.4 Velocities

$$v_{\rm phys}[{\rm km/s}] = v_{\rm code}[{\rm km}\sqrt{a}\,/\,{\rm s}] \times \sqrt{a} \tag{6}$$

3.5 Volume and Effective Cell Size

$$V_{\rm phys}[{\rm kpc^3}] = V_{\rm code}[({\rm ckpc}/h)^3] \times \left(\frac{a}{h}\right)^3 \tag{7}$$

The effective spherical radius of a Voronoi cell is:

$$r_{\rm eff} = \left(\frac{3\, V_{\rm phys}}{4\pi}\right)^{1/3} \tag{8}$$

This is used by the resolution-matching refinement criterion (§5.3.2).

4. ISM Treatment: Star-Forming Cells

In Illustris/TNG, gas cells with SFR > 0 are placed on an effective equation of state (EOS) that represents the unresolved multiphase interstellar medium [6]. Their `InternalEnergy` encodes an effective pressure, not the actual ISM temperature. The computed temperature for these cells can be $\sim 10^5$–10^7 K, which is unphysical for the ISM.

The converter provides three treatments via `--sfr-treatment`:

- `cap-temperature` (default): cap T at a user-specified value (default 2×10^4 K) for all cells with SFR > 0. This is the most common approach in Lyα RT post-processing [7].
- `exclude`: remove all star-forming cells. This is more aggressive but avoids any ambiguity.
- `as-is`: use the computed temperature without modification. Not recommended for Lyα RT but included for completeness.

5. Conversion to Octree AMR

LaRT uses an octree AMR grid for photon propagation. The Voronoi mesh of Illustris is unstructured and cannot be used directly. The converter builds an adaptive octree and assigns physical quantities from the Voronoi data using the following steps.

5.1 Step 1: Data Loading and Unit Conversion

The converter reads `PartType0` fields from the input HDF5 file (either a snapshot chunk or an API cutout) and converts all quantities to physical units as described in Section 3..
Coordinates are recentered so that the galaxy center is at the origin. If `--center` is not specified, the centroid of the gas cell distribution is used. If `--boxsize` is not specified, the bounding box of the data is computed with a 10% margin.

5.2 Step 2: KD-Tree Construction

A KD-tree [8] is built from the Voronoi generator positions using `scipy.spatial.cKDTree`. This tree enables $O(\log N)$ nearest-neighbor queries from any point in the domain.

5.3 Step 3: Adaptive Octree Refinement

The octree is built using the `AMRGrid` class from `AMR_grid.py`. Two refinement criteria are available, applied in sequence:

5.3.1 Density and Velocity Gradient Refinement

A cell is refined if the relative variation across its sub-cell probe points exceeds a threshold:

$$\delta = \frac{q_{\max} - q_{\min}}{q_{\max} + q_{\min} + \epsilon} > \delta_{\text{thresh}} \tag{9}$$

where q is the density (or velocity magnitude), and ϵ is a small floor to prevent division by zero. The default threshold is $\delta_{\text{thresh}} = 0.3$. The density and velocity criteria are combined with logical OR: a cell is refined if *either* criterion is satisfied.
All cells are first uniformly refined to a minimum level (`--level-min`, default 3) before the gradient criterion is applied.

5.3.2 Resolution-Matching Refinement

If `--match-resolution` is specified, an octree cell is additionally refined whenever its half-width exceeds a factor f (default 1.0) times the effective Voronoi cell radius $r_{\rm eff}$ (Equation 8) of the nearest Voronoi cell:

$$h_{\rm cell} > f \times r_{\rm eff,nearest} \tag{10}$$

This ensures the octree resolution matches or exceeds the native Voronoi resolution of the simulation.

5.4 Step 4: Property Assignment via Nearest-Neighbor

Physical quantities ($n_{\rm H}$, T, v_x, v_y, v_z) are assigned to each octree leaf cell by querying the KD-tree for the nearest Voronoi generator and copying its values.

This is the **physically exact** assignment for Voronoi tessellations: by definition, the Voronoi cell of generator $\mathbf{g}_i$ is the set of all points closer to $\mathbf{g}_i$ than to any other generator. The KD-tree nearest-neighbor query therefore recovers the correct cell ownership.

Note: Kernel smoothing (as appropriate for SPH data) is *not* used because it would blur the sharp density and velocity discontinuities that the Voronoi mesh preserves (e.g., contact discontinuities, shock fronts).

5.5 Step 5: Optional Physics Computation

When `--compute-physics` is specified, the following optional columns are computed and appended to the output:

- **Neutral fraction** ($x_{\rm HI}$): from TNG native `NeutralHydrogenAbundance` (`--ionization from_illustris`), or computed via the CIE formula (`--ionization cie`), or set to 1 (`--ionization full_neutral`).
- **Electron density** (n_e): from TNG `ElectronAbundance` (if using `from_illustris`), or $n_e = n_{\rm H}(1-x_{\rm HI})$.
- **Emissivity** (controlled by `--emissivity-line`):
  - `lya` (default): Case B Lyα recombination plus collisional excitation.
  - `civ`: CIE C IV $\lambda\lambda$1548,1550 collisional excitation. Ion fraction from Gnat & Sternberg (2007), collision strength $\Omega_{\rm eff} = 7.5$ from Aggarwal & Keenan (2004), C/H solar abundance from Asplund et al. (2009). Requires metallicity.
  - `ovi`: CIE O VI $\lambda\lambda$1032,1038 collisional excitation. Ion fraction from Gnat & Sternberg (2007), collision strength $\Omega_{\rm eff} = 3.8$ from Aggarwal & Keenan (2004), O/H solar abundance from Asplund et al. (2009). Requires metallicity.

  The CIE metal-line emissivity is: $\varepsilon = n_{\rm ion} \times n_e \times q(T)$ [photons cm^{-3} s^{-1}], where $n_{\rm ion} = n_{\rm H}\,(Z/Z_\odot) A_X\, f_{\rm ion}(T)$ and $q(T) = 8.629 \times 10^{-6}/(g_l\sqrt{T})\,\Omega\,\exp(-\Delta E/kT)$. CIE is appropriate for hot gas ($T \gtrsim 10^5$ K) where these ions peak; for cooler gas the UV background dominates and PIE tables would be more accurate.
- **Dust density**: Laursen et al. (2009) [7] prescription: $n_{\rm dust} = (Z/Z_{\rm ref}) \times (n_{\rm HI} + f_{\rm ion}\, n_{\rm HII})$ with $Z_{\rm ref} = 0.0134$, $f_{\rm ion} = 0.01$.
- **Metallicity**: per-cell Z from `GFM_Metallicity`.

5.6 Step 6: Output

The converter writes the standard LaRT generic AMR format (HDF5, FITS, or plain text), identical to the output of `convert_ramses_to_generic.py`. See the RAMSES data structure document for the full schema specification.

Dataset	Description
`x, y, z`	Octree leaf-cell center [kpc, centered]
`level`	AMR refinement level
`gasDen`	$n_{\rm H}$ [cm^{-3}]
`T`	Temperature [K]
`vx, vy, vz`	Velocity [km/s]
`metallicity`	Metal mass fraction Z (optional)
`xHI`	Neutral fraction $x_{\rm HI}$ (optional)
`n_e`	Electron density [cm^{-3}] (optional)
`emissivity`	Line emissivity [cm^{-3} s^{-1}] (optional; Lyα, C IV, or O VI)
`ndust`	Dust pseudo-number density [cm^{-3}] (optional)

6. Converter Usage

The converter script is located at:

```
LaRT_v2.00/python/AMR_grid/convert_illustris_to_generic.py
```

6.1 Basic Usage (Local File)

```
python convert_illustris_to_generic.py cutout.hdf5 \
    -o galaxy.h5 \
    --output-unit kpc \
    --level-max 8 \
    --compute-physics \
    --ionization from_illustris \
    --plot
```

6.2 API Download Mode

The converter can download subhalo cutouts from the IllustrisTNG public API. A free API key is required (register at `tng-project.org/users/register/`).

```
python convert_illustris_to_generic.py \
    --api-key YOUR_API_KEY \
    --simulation TNG50-1 \
    --snap 99 \
```

```
    --subhalo-id 0 \
    -o galaxy.h5 \
    --compute-physics
```

6.3 Full Command-Line Reference

Flag	Default	Description
`input_file`	(positional)	Input HDF5 file (cutout or snapshot chunk)
`-o, --output`	(required)	Output file (.h5, .fits.gz, or .dat)
`--output-unit`	kpc	Unit of output coordinates
Octree refinement		
`--level-max`	8	Maximum AMR level
`--level-min`	3	Minimum uniform refinement level
`--dens-threshold`	0.3	Density gradient threshold $\delta_{\rm thresh}$
`--vel-threshold`	0.3	Velocity gradient threshold
`--match-resolution`	off	Enable resolution-matching refinement
`--resolution-factor`	1.0	Factor f for resolution matching
Physics		
`--compute-physics`	off	Compute $x_{\rm HI}$, n_e, emissivity, $n_{\rm dust}$
`--ionization`	`from_illustris`	Source of $x_{\rm HI}$: `from_illustris`, `cie`, `full_neutral`, or `none`
`--emissivity-line`	`lya`	Emission line for emissivity column: `lya` (Case B Lyα), `civ` (CIE C IV 1548+1550), `ovi` (CIE O VI 1032+1038)
ISM treatment		
`--sfr-treatment`	`cap-temperature`	SFR > 0 cells: `cap-temperature`, `exclude`, or `as-is`
`--sfr-cap-temp`	2×10^4	Temperature cap [K]
Geometry		
`--center X Y Z`	auto	Galaxy center [physical kpc]
`--boxsize`	auto	Box side length [kpc]
API download (optional)		
`--api-key`	—	TNG API key
`--simulation`	TNG50-1	Simulation name
`--snap`	99	Snapshot number
`--subhalo-id`	0	Subhalo ID

Flag	Default	Description
Miscellaneous		
`--plot`	off	Generate diagnostic slice plot (nH and T)

6.4 Suggested LaRT Input

After conversion, the converter prints a suggested LaRT input file snippet. A typical configuration:

```
&parameters
 par%use_amr_grid    = .true.
 par%amr_type        = 'generic'
 par%amr_file        = 'galaxy.h5'
 par%distance_unit   = 'kpc'
 par%no_photons      = 1e+06
 par%spectral_type   = 'voigt'
 par%source_geometry = 'diffuse_emissivity'
 par%nxfreq          = 121
 par%nxim            = 100
 par%nyim            = 100
/
```

With `par%source_geometry = 'diffuse_emissivity'`, LaRT uses the `emissivity` column from the converted file to determine the photon source distribution. Alternatively, use `'point'` for a central point source.

7. End-to-End Workflow

A typical analysis workflow is:

1. **Obtain data**: download a subhalo cutout from the TNG public API, or locate a local snapshot chunk file.
2. **Convert to octree**:

```
python convert_illustris_to_generic.py cutout.hdf5 \
    -o galaxy.h5 --output-unit kpc \
    --level-max 8 --compute-physics \
    --ionization from_illustris \
    --sfr-treatment cap-temperature \
    --boxsize 200 --plot
```

3. **Inspect the grid**: use `AMRGrid.read` and `slice_plot` to verify the density and velocity structure.
4. **Run LaRT**:

```
mpirun -np 64 LaRT.x galaxy_lya.in
```

5. **Analyze output**: use `read_lart.py` to read the output and produce spectra, surface-brightness maps, etc.

8. Technical Notes

8.1 Why Nearest-Neighbor, Not Kernel Smoothing

For Voronoi data from Arepo, the nearest-neighbor KD-tree query recovers the exact Voronoi cell assignment. Each point in space belongs to exactly one cell (the one whose generator is closest), so the nearest-neighbor query returns the correct cell-averaged quantities.

Kernel smoothing is appropriate for SPH simulations where particle quantities represent overlapping kernel-weighted contributions. Applying kernel smoothing to Voronoi data would artificially blur the sharp density and velocity discontinuities that the moving mesh is designed to capture (e.g., contact discontinuities, shock fronts, galactic outflow boundaries).

8.2 Memory and Performance

For a typical TNG cutout with $\sim 10^6$ gas cells:

- KD-tree construction: ~ 1 GB, ~ 10 s
- Octree refinement to level 8: ~ 1–5×10^6 leaf cells, ~ 1–5 min
- Output HDF5 file: ~ 50–200 MB

For `--level-max` > 9, the octree can contain $> 10^8$ cells and may require > 10 GB of memory. Use with caution.

8.3 Coordinate Centering

API cutout files from TNG contain cells from the entire parent halo, not centered on the subhalo. The converter automatically recenters the data by computing the centroid of the gas cell distribution. For more precise centering (e.g., on the subhalo potential minimum), use `--center X Y Z` with the subhalo position from the group catalog.

8.4 Comparison with RAMSES Converter

The Illustris and RAMSES converters produce the same output format and can be used interchangeably with LaRT. The key differences are:

	RAMSES	Illustris
Input mesh	Octree AMR	Voronoi (unstructured)
Conversion	Direct: read leaf cells	Resample: build new octree
Fortran converter	Yes	No (Python only)
Ionization	CIE only	CIE or TNG native
Temperature	Directly from thermal pressure	From InternalEnergy + x_e
ISM treatment	Not needed	SFR temperature cap

9. Data Access

9.1 Public API

Both projects provide REST APIs for querying and downloading data:

- Illustris: `https://www.illustris-project.org/api/`
- IllustrisTNG: `https://www.tng-project.org/api/`

An API key is required (free registration for academic use). The API supports searching subhalos by mass, SFR, or other properties, and downloading cutout HDF5 files for individual halos or subhalos.

9.2 Direct File Downloads

Full snapshot files can be downloaded directly from the project websites. A single TNG100-1 snapshot at $z = 0$ is ~ 20 GB (split across ~ 450 chunk files).

9.3 Python Helper Scripts

The official `illustris_python` package (`https://github.com/illustristng/illustris_python`) provides convenience functions for loading subhalos from local snapshot files. The LaRT converter does *not* depend on this package; it reads HDF5 files directly with `h5py`.

References

[1] Vogelsberger, M., Genel, S., Springel, V., et al. 2014, MNRAS, 444, 1518 (*Introducing the Illustris simulation: The evolving population of galaxies across cosmic time*).

[2] Nelson, D., Pillepich, A., Genel, S., et al. 2015, Astron. Comput., 13, 12 (*The Illustris simulation: Public data release*).

[3] Nelson, D., Springel, V., Pillepich, A., et al. 2019, Comput. Astrophys. Cosmol., 6, 2 (*The IllustrisTNG simulations: Public data release*).

[4] Pillepich, A., Springel, V., Nelson, D., et al. 2018, MNRAS, 473, 4077 (*Simulating galaxy formation with the IllustrisTNG model*).

[5] Springel, V. 2010, MNRAS, 401, 791 (*E pur si muove: Galilean-invariant cosmological hydrodynamical simulations on a moving mesh*).

[6] Springel, V., Hernquist, L. 2003, MNRAS, 339, 289 (*Cosmological smoothed particle hydrodynamics simulations: A hybrid multiphase model for star formation*). Effective EOS model for the multiphase ISM.

[7] Laursen, P., Sommer-Larsen, J., Andersen, A. C. 2009, ApJ, 704, 1640 (*Lyman-α radiative transfer with dust*). Source of the dust prescription and the ISM temperature-cap convention.

[8] Bentley, J. L. 1975, Commun. ACM, 18, 509 (*Multidimensional binary search trees used for associative searching*).